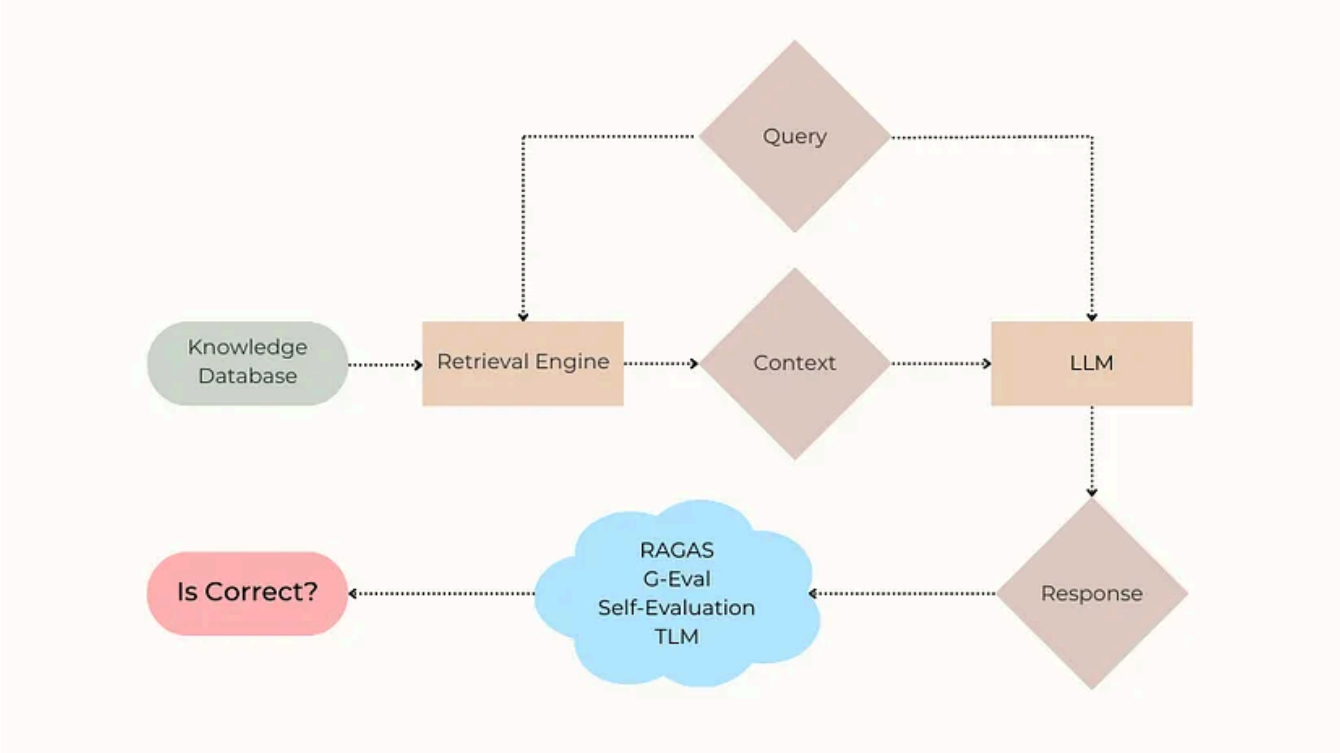
# Benchmarking Hallucination Detection Methods in RAG

Evaluating methods to enhance reliability in LLM-generated responses.

Hui Wen Goh · Follow
Published in Towards Data Science
9 min read · Sep 9, 2024



Unchecked hallucination remains a big problem in today's Retrieval-Augmented Generation applications. This study evaluates popular hallucination detectors across 4 public RAG datasets. Using AUROC and precision/recall, we report *how well* methods like G-eval, Ragas, and the Trustworthy Language Model are able to **automatically flag incorrect LLM responses.**




Using various hallucination detection methods to identify LLM errors in RAG systems.

I am currently working as a Machine Learning Engineer at Cleanlab, where I have contributed to the development of the Trustworthy Language Model discussed in this article. I am excited to present this method and evaluate it alongside others in the following benchmarks.

## The Problem: Hallucinations and Errors in RAG Systems

Large Language Models (LLM) are known to *hallucinate* incorrect answers when asked questions not well-supported within their training data. Retrieval Augmented Generation (RAG) systems mitigate this by augmenting the LLM with the ability to *retrieve* context and information from a specific knowledge database. While organizations are quickly adopting RAG to pair the power of LLMs with their own proprietary data, hallucinations and logical errors remain a big problem. In one highly publicized case, a major airline (Air Canada) lost a court case after their RAG chatbot hallucinated important details of their refund policy.

To understand this issue, let's first revisit how a RAG system works. When a user asks a question ( `"Is this is refund eligible?"` ), the *retrieval* component searches the knowledge database for relevant information needed to respond accurately. The most relevant search results are formatted into a *context* which is fed along with the user's question into a LLM that *generates* the response presented to the user. Because enterprise RAG systems are often complex, the final response might be incorrect for many reasons including:

1. LLMs are brittle and prone to hallucination. Even when the retrieved context contains the correct answer within it, the LLM may fail to generate an accurate response, especially if synthesizing the response requires reasoning across different facts within the context.

2. The retrieved context may not contain information required to accurately respond, due to suboptimal search, poor document chunking/formatting, or the absence of this information within the knowledge database. In such cases, the LLM may still attempt to answer the question and hallucinate an incorrect response.

While some use the term *hallucination* to refer only to specific types of LLM errors, here we use this term synonymously with *incorrect response.* What matters to the users of your RAG system is the accuracy of its answers and being able to trust them. Unlike RAG benchmarks that assess many system properties, we exclusively

study: **how effectively different detectors could alert your RAG users when the answers are incorrect.**

A RAG answer might be incorrect due to problems during *retrieval* or *generation*. Our study focuses on the latter issue, which stems from the fundamental unreliability of LLMs.

## The Solution: Hallucination Detection Methods

Assuming an existing retrieval system has fetched the *context* most relevant to a user's question, we consider algorithms to detect when **the LLM response generated based on this context should not be trusted**. Such hallucination detection algorithms are critical in high-stakes applications spanning medicine, law, or finance. Beyond flagging untrustworthy responses for more careful human review, such methods can be used to determine when it is worth executing more expensive retrieval steps (e.g. searching additional data sources, rewriting queries, etc).

Here are the hallucination detection methods considered in our study, all based on using LLMs to evaluate a generated response:

**Self-evaluation ("Self-eval")** is a simple technique whereby the LLM is asked to evaluate the generated answer and rate its confidence on a scale of 1–5 (Likert scale). We utilize *chain-of-thought* (CoT) prompting to improve this technique, asking the LLM to explain its confidence before outputting a final score. Here is the specific prompt template used:

> Question: {question}
> Answer: {response}
>
> *Evaluate how confident you are that the given Answer is a good and accurate response to the Question.*
> *Please assign a Score using the following 5-point scale:*
> *1: You are not confident that the Answer addresses the Question at all, the Answer may be entirely off-topic or irrelevant to the Question.*
> *2: You have low confidence that the Answer addresses the Question, there are doubts and uncertainties about the accuracy of the Answer.*
> *3: You have moderate confidence that the Answer addresses the Question, the Answer seems reasonably accurate and on-topic, but with room for improvement.*
> *4: You have high confidence that the Answer addresses the Question, the Answer provides accurate information that addresses most of the Question.*

> *5: You are extremely confident that the Answer addresses the Question, the Answer is highly accurate, relevant, and effectively addresses the Question in its entirety.*
>
> *The output should strictly use the following template: Explanation: [provide a brief reasoning you used to derive the rating Score] and then write 'Score: <rating>' on the last line.*

**G-Eval** (from the DeepEval package) is a method that uses CoT to automatically develop multi-step criteria for assessing the quality of a given response. In the G-Eval paper (Liu et al.), this technique was found to correlate with Human Judgement on several benchmark datasets. Quality can be measured in various ways specified as a LLM prompt, here we specify it should be assessed based on the factual correctness of the response. Here is the criteria that was used for the G-Eval evaluation:

> Determine whether the output is factually correct given the context.

**Hallucination Metric** (from the DeepEval package) estimates the likelihood of hallucination as the degree to which the LLM response contradicts/disagrees with the context, as assessed by another LLM.

**RAGAS** is a RAG-specific, LLM-powered evaluation suite that provides various scores which can be used to detect hallucination. We consider each of the following RAGAS scores, which are produced by using LLMs to estimate the requisite quantities:

1. Faithfulness — The fraction of claims in the answer that are supported by the provided context.
2. Answer Relevancy is the mean cosine similarity of the vector representation to the original question with the vector representations of three LLM-generated questions from the answer. Vector representations here are embeddings from the `BAAI/bge-base-en encoder`.
3. Context Utilization measures to what extent the context was relied on in the LLM response.

**Trustworthy Language Model (TLM)** is a model uncertainty-estimation technique that evaluates the trustworthiness of LLM responses. It uses a combination of self-reflection, consistency across multiple sampled responses, and probabilistic

measures to identify errors, contradictions and hallucinations. Here is the prompt template used to prompt TLM:

> Answer the QUESTION using information only from
> CONTEXT: {context}
> QUESTION: {question}

## Evaluation Methodology

We will compare the hallucination detection methods stated above across 4 public Context-Question-Answer datasets spanning different RAG applications.

For each user *question* in our benchmark, an existing retrieval system returns some relevant *context*. The user query and context are then input into a *generator* LLM (often along with an application-specific system prompt) in order to generate a response for the user. Each detection method takes in the {user query, retrieved context, LLM response} and returns a score between 0–1, indicating the likelihood of hallucination.

To evaluate these hallucination detectors, we consider how reliably these scores take lower values when the LLM responses are incorrect vs. being correct. In each of our benchmarks, there exist ground-truth annotations regarding the correctness of each LLM response, which we solely reserve for evaluation purposes. We evaluate hallucination detectors based on **AUROC**, defined as the probability that their score will be lower for an example drawn from the subset where the LLM responded incorrectly than for one drawn from the subset where the LLM responded correctly. Detectors with greater AUROC values can be used to **catch RAG errors in your production system with greater precision/recall**.

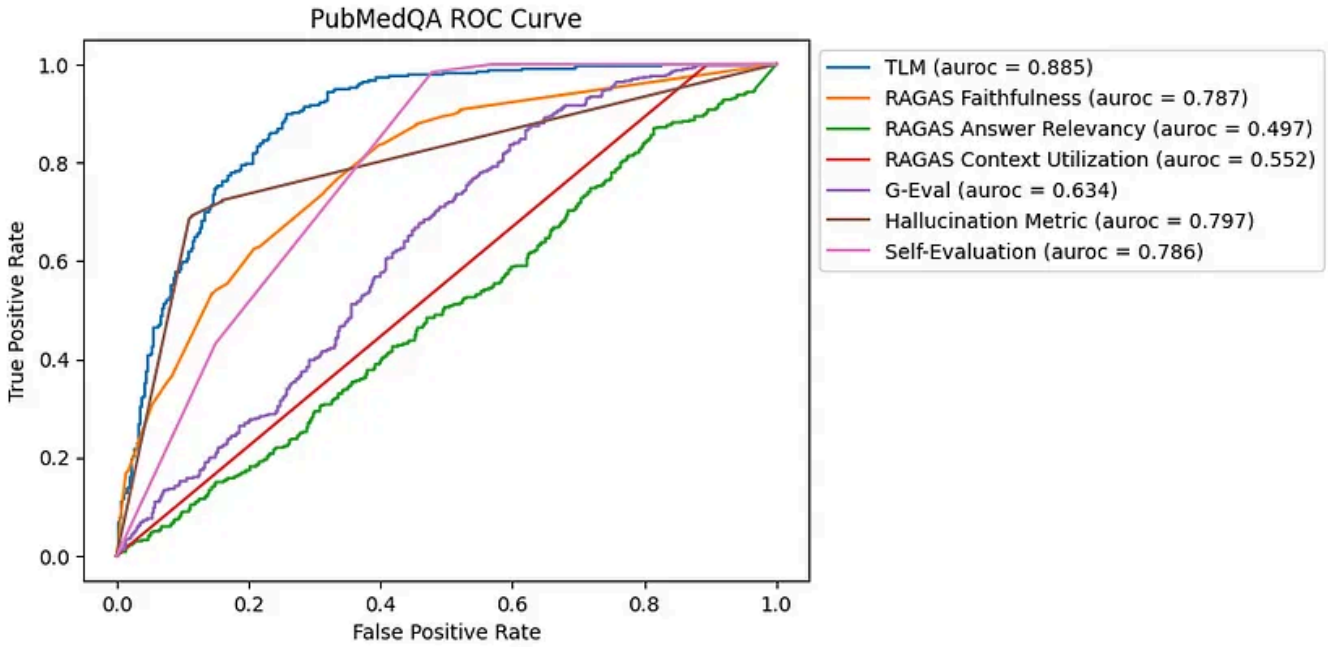
All of the considered hallucination detection methods are themselves powered by a LLM. For fair comparison, we fix this LLM model to be `gpt-4o-mini` across all of the methods.

## Benchmark Results

We describe each benchmark dataset and the corresponding results below. These datasets stem from the popular HaluBench benchmark suite (we do not include the other two datasets from this suite, as we discovered significant errors in their ground truth annotations).

## PubMedQA

PubMedQA is a biomedical Q&A dataset based on PubMed abstracts. Each instance in the dataset contains a passage from a PubMed (medical publication) abstract, a question derived from passage, for example: `Is a 9-month treatment sufficient in tuberculous enterocolitis?` , and a generated answer.

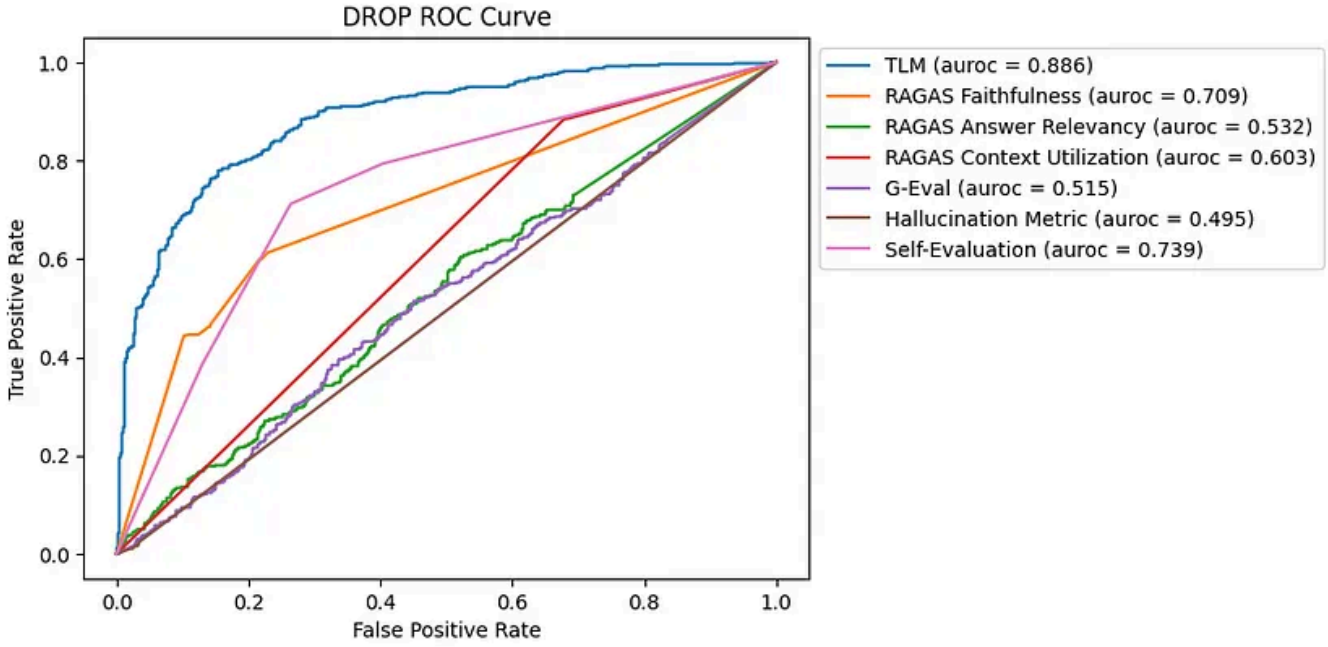



ROC Curve for PubMedQA Dataset

In this benchmark, TLM is the most effective method for discerning hallucinations, followed by the Hallucination Metric, Self-Evaluation and RAGAS Faithfulness. Of the latter three methods, RAGAS Faithfulness and the Hallucination Metric were more effective for catching incorrect answers with high precision (RAGAS Faithfulness had an average precision of `0.762` , Hallucination Metric had an average precision of `0.761` , and Self-Evaluation had an average precision of `0.702` ).

## DROP

DROP, or "Discrete Reasoning Over Paragraphs", is an advanced Q&A dataset based on Wikipedia articles. DROP is difficult in that the questions require reasoning over context in the articles as opposed to simply extracting facts. For example, given context containing a Wikipedia passage describing touchdowns in a Seahawks vs. 49ers Football game, a sample question is: `How many touchdown runs measured 5-yards or less in total yards?` , requiring the LLM to read each touchdown run and then compare the length against the 5-yard requirement.



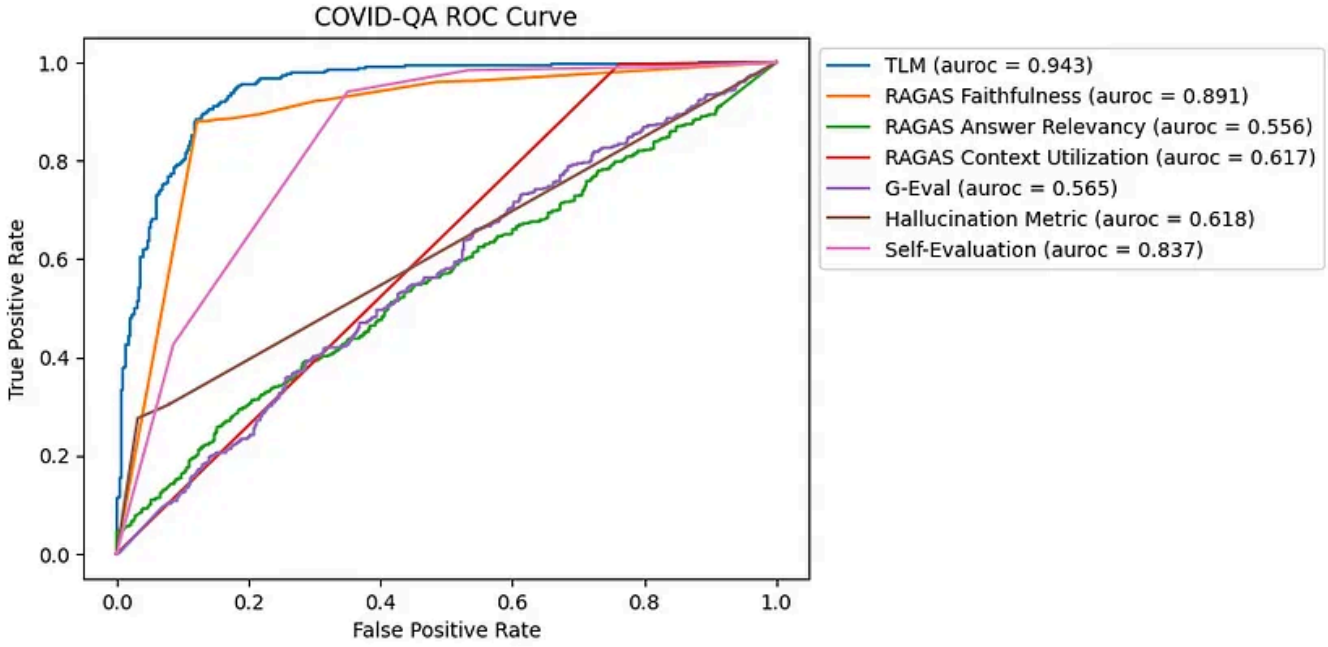

ROC Curve for DROP Dataset

Most methods faced challenges in detecting hallucinations in this DROP dataset due to the complexity of the reasoning required. TLM emerges as the most effective method for this benchmark, followed by Self-Evaluation and RAGAS Faithfulness.

## COVID-QA

COVID-QA is a Q&A dataset based on scientific articles related to COVID-19. Each instance in the dataset includes a scientific passage related to COVID-19 and a question derived from the passage, for example: `How much similarity the SARS-COV-2 genome sequence has with SARS-COV?`

Compared to DROP, this is a simpler dataset as it only requires basic synthesis of information from the passage to answer more straightforward questions.

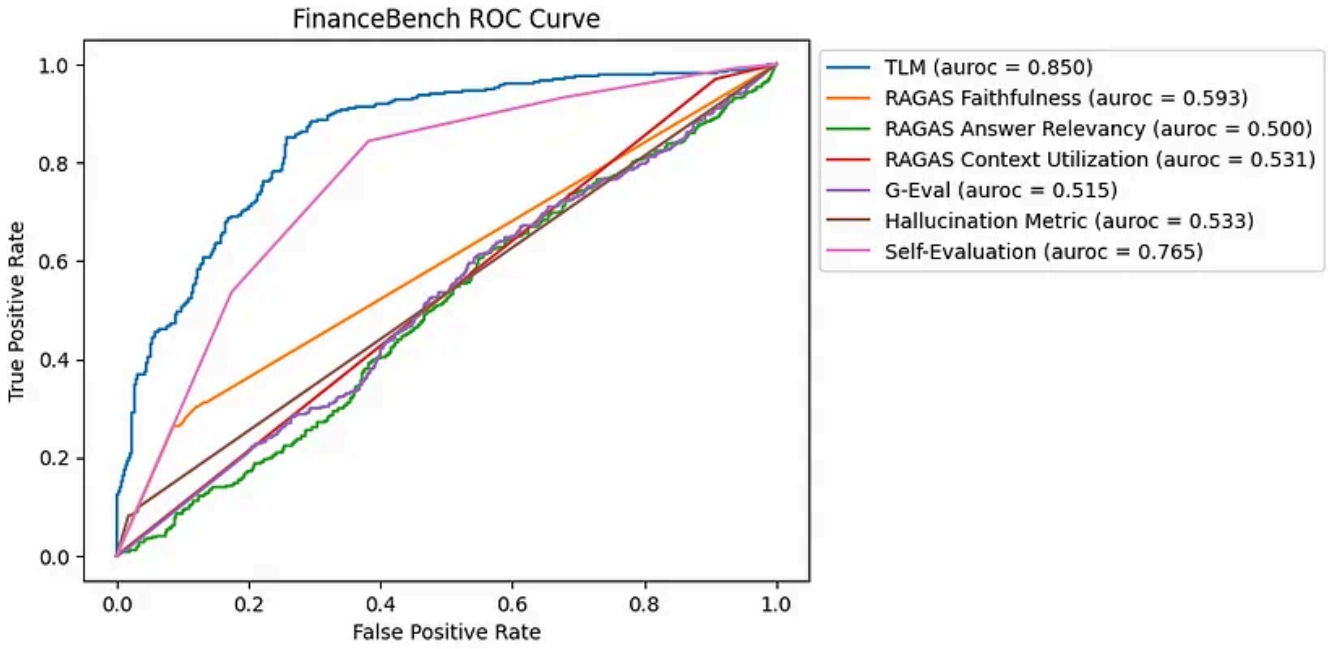



ROC Curve for COVID-QA Dataset

In the COVID-QA dataset, TLM and RAGAS Faithfulness both exhibited strong performance in detecting hallucinations. Self-Evaluation also performed well, however other methods, including RAGAS Answer Relevancy, G-Eval, and the Hallucination Metric, had mixed results.

## FinanceBench

FinanceBench is a dataset containing information about public financial statements and publicly traded companies. Each instance in the dataset contains a large retrieved context of plaintext financial information, a question regarding that information, for example: `What is FY2015 net working capital for Kraft Heinz?`, and a numeric answer like: `$2850.00`.




ROC Curve for FinanceBench Dataset

For this benchmark, TLM was the most effective in identifying hallucinations, followed closely by Self-Evaluation. Most other methods struggled to provide significant improvements over random guessing, highlighting the challenges in this dataset that contains large amounts of context and numerical data.

## Discussion

Our evaluation of hallucination detection methods across various RAG benchmarks reveals the following key insights:

1. **Trustworthy Language Model (TLM)** consistently performed well, showing strong capabilities in identifying hallucinations through a blend of self-reflection, consistency, and probabilistic measures.

2. **Self-Evaluation** showed consistent effectiveness in detecting hallucinations, particularly effective in simpler contexts where the LLM's self-assessment can be accurately gauged. While it may not always match the performance of TLM, it remains a straightforward and useful technique for evaluating response quality.

3. **RAGAS Faithfulness** demonstrated robust performance in datasets where the accuracy of responses is closely linked to the retrieved context, such as in PubMedQA and COVID-QA. It is particularly effective in identifying when claims in the answer are not supported by the provided context. However, its effectiveness was variable depending on the complexity of the questions. By default, RAGAS uses `gpt-3.5-turbo-16k` for generation and `gpt-4` for the critic

LLM, which produced worse results than the RAGAS with `gpt-4o-mini` results we reported here. RAGAS failed to run on certain examples in our benchmark due to its sentence parsing logic, which we fixed by appending a period (.) to the end of answers that did not end in punctuation.

4. **Other Methods** like G-Eval and Hallucination Metric had mixed results, and exhibited varied performance across different benchmarks. Their performance was less consistent, indicating that further refinement and adaptation may be needed.

Overall, TLM, RAGAS Faithfulness, and Self-Evaluation stand out as more reliable methods to detect hallucinations in RAG applications. For high-stakes applications, combining these methods could offer the best results. Future work could explore hybrid approaches and targeted refinements to better conduct hallucination detection with specific use cases. By integrating these methods, RAG systems can achieve greater reliability and ensure more accurate and trustworthy responses.

*Unless otherwise noted, all images are by the author.*

Generative Ai Tools | Retrieval Augmented Gen | Genai | Large Language Models | Llm

Follow

## Written by Hui Wen Goh

138 Followers · Writer for Towards Data Science

Machine Learning Engineer at Cleanlab, focused on enhancing data and LLM reliability to improve the dependability of ML and AI applications.

## More from Hui Wen Goh and Towards Data Science

Shaw Talebi in Towards Data Science

### 5 AI Projects You Can Build This Weekend (with Python)

From beginner-friendly to advanced

Oct 9 3.2K 53

Mauro Di Pietro in Towards Data Science

## GenAI with Python: Build Agents from Scratch (Complete Tutorial)

with Ollama, LangChain, LangGraph (No GPU, No APIKEY)

Sep 29 1.7K 24

Thuwarakesh Murallie in Towards Data Science

## I Fine-Tuned the Tiny Llama 3.2 1B to Replace GPT-4o

Is the fine-tuning effort worth more than few-shot prompting?

6d ago 1.7K 19

Hesam Sheikh in Towards Data Science

## How I Studied LLMs in Two Weeks: A Comprehensive Roadmap

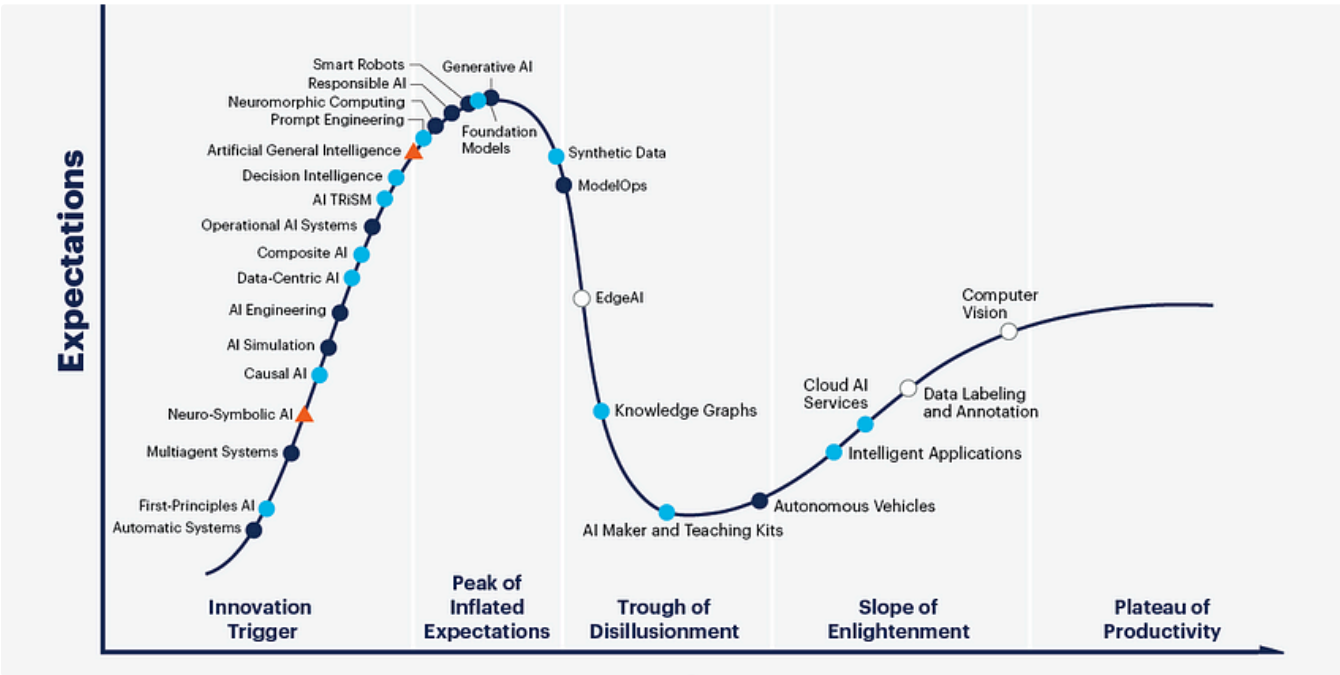
A day-by-day detailed LLM roadmap from beginner to advanced, plus some study tips

3d ago 824 5

See all from Hui Wen Goh

See all from Towards Data Science

## Recommended from Medium

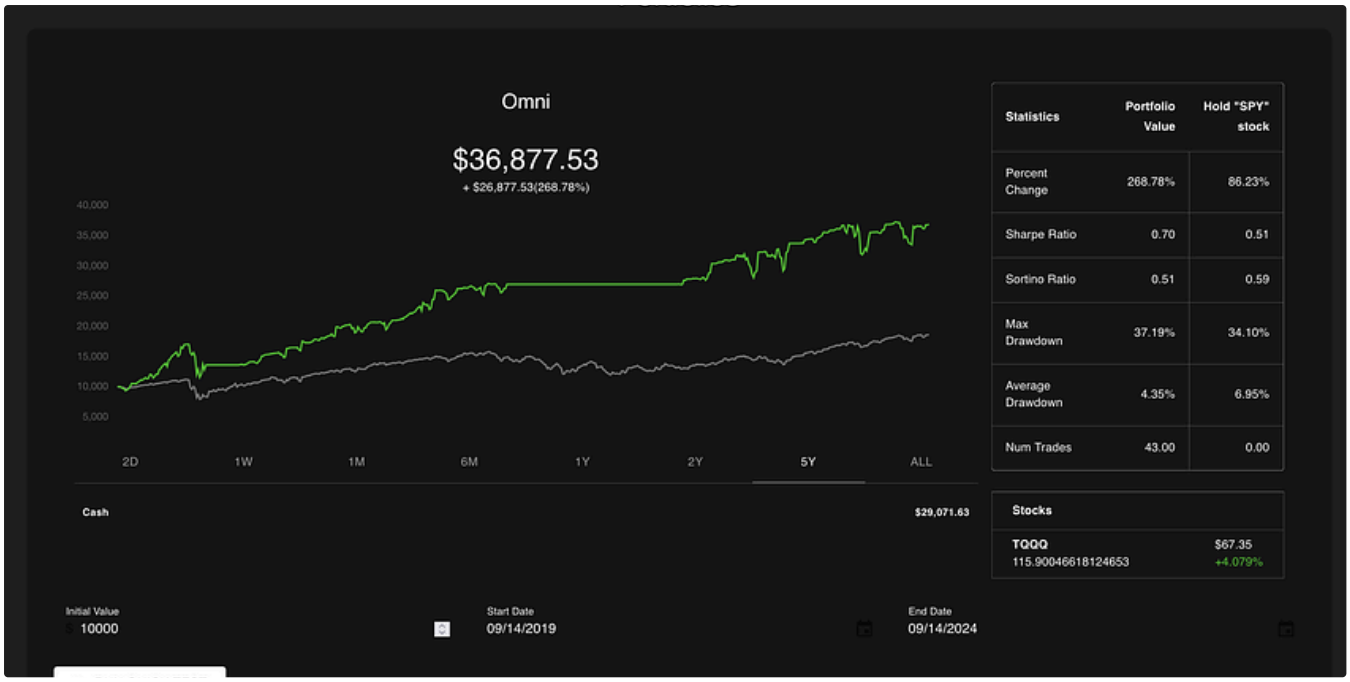



Vishal Rajput in AIGuys

## Why GEN AI Boom Is Fading And What's Next?

Every technology has its hype and cool down period.

Sep 4 2.3K 72




Austin Starks in DataDrivenInvestor

# I used OpenAI's o1 model to develop a trading strategy. It is DESTROYING the market

It literally took one try. I was shocked.

Sep 15 4.3K 119

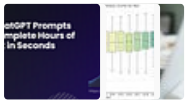
## Lists

### Natural Language Processing

1766 stories · 1367 saves

### AI Regulation

6 stories · 593 saves

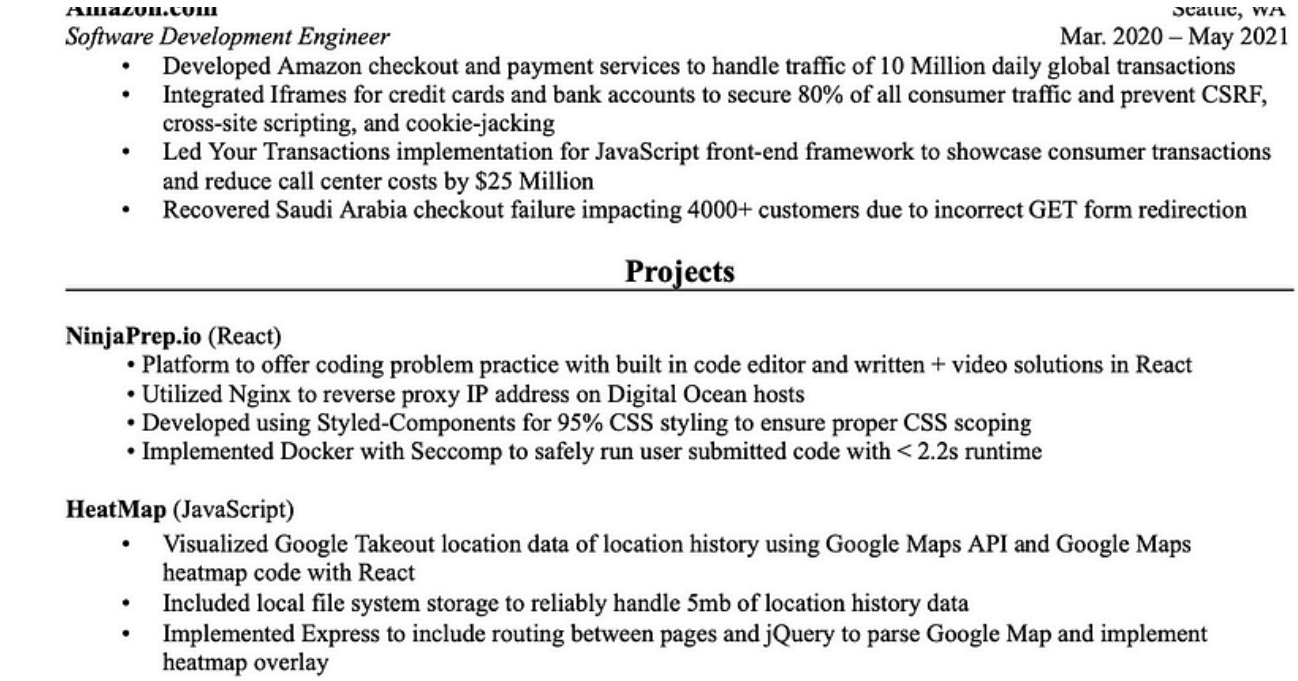


### ChatGPT prompts

50 stories · 2121 saves

### Generative AI Recommended Reading

52 stories · 1446 saves



Amazon.com Seattle, WA
*Software Development Engineer* Mar. 2020 – May 2021

- Developed Amazon checkout and payment services to handle traffic of 10 Million daily global transactions
- Integrated Iframes for credit cards and bank accounts to secure 80% of all consumer traffic and prevent CSRF, cross-site scripting, and cookie-jacking
- Led Your Transactions implementation for JavaScript front-end framework to showcase consumer transactions and reduce call center costs by $25 Million
- Recovered Saudi Arabia checkout failure impacting 4000+ customers due to incorrect GET form redirection

**Projects**

**NinjaPrep.io** (React)
- Platform to offer coding problem practice with built in code editor and written + video solutions in React
- Utilized Nginx to reverse proxy IP address on Digital Ocean hosts
- Developed using Styled-Components for 95% CSS styling to ensure proper CSS scoping
- Implemented Docker with Seccomp to safely run user submitted code with < 2.2s runtime

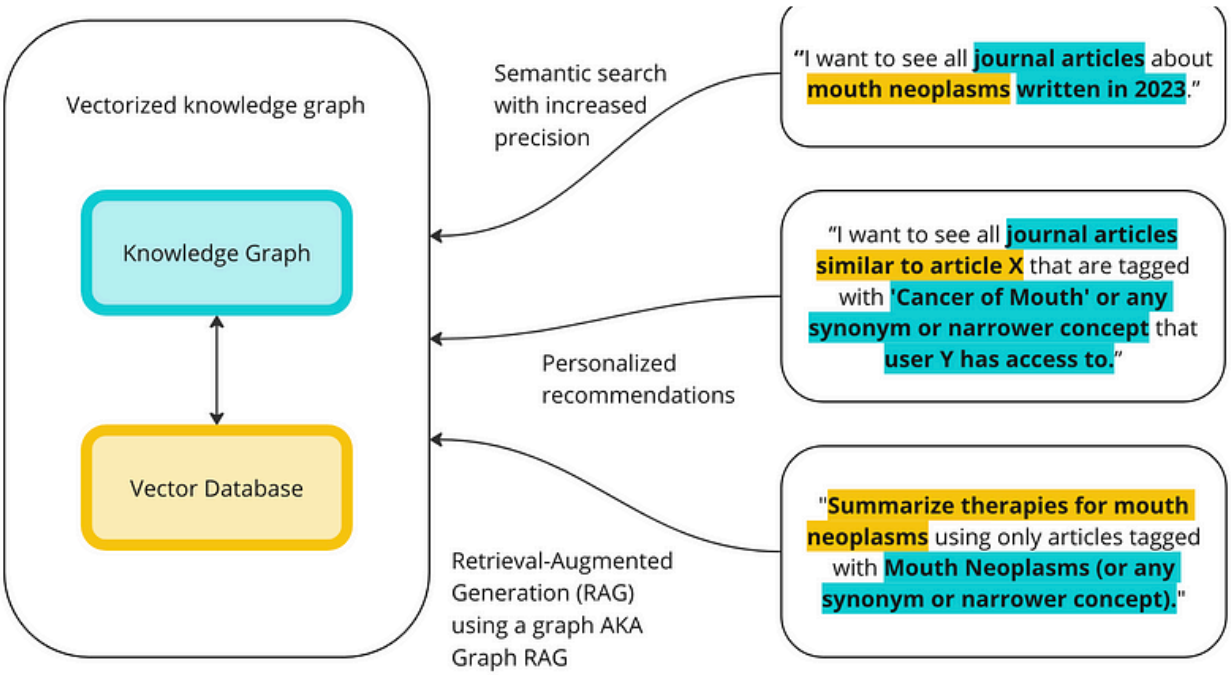
**HeatMap** (JavaScript)
- Visualized Google Takeout location data of location history using Google Maps API and Google Maps heatmap code with React
- Included local file system storage to reliably handle 5mb of location history data
- Implemented Express to include routing between pages and jQuery to parse Google Map and implement heatmap overlay

Alexander Nguyen in Level Up Coding

## The resume that got a software engineer a $300,000 job at Google.

1-page. Well-formatted.

Jun 1 24K 487

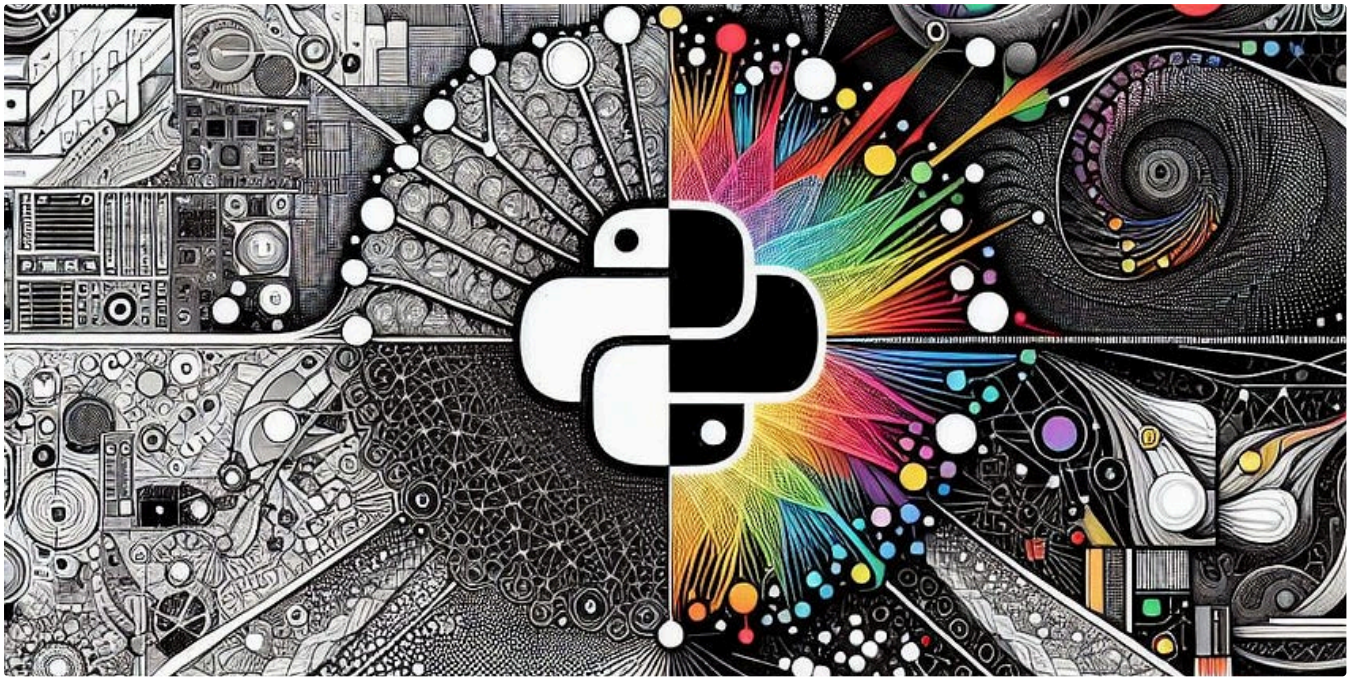



Steve Hedden in Towards Data Science

## How to Implement Graph RAG Using Knowledge Graphs and Vector Databases

A Step-by-Step Tutorial on Implementing Retrieval-Augmented Generation (RAG), Semantic Search, and Recommendations

Sep 6 1.4K 18



Muhammad Saad Uddin in AI Advances

## Why Python 3.13 Release Could Be a Game Changer for AI and ML

Discover How It Will Transform ML and AI Dynamics

Oct 11 1.6K 12


Emmanuel Ikogho

## Data Science is dying; here's why

Why 85% of data science projects fail

Sep 3 1.6K 78

See more recommendations